PRODUCTION MASTERPLAN

Helm for Kubernetes: YouTube Creator Blueprint

A Complete Production Roadmap, Screencast Architecture, Pedagogical Framework, and Packaging Strategy

COURSE DURATION	TARGET AUDIENCE	TARGET RETENTION	LOCAL LAB STACK
60–80 Mins	**DevOps & Cloud Devs**	**44%+ AVD**	**Kind + Helm 3**
Single Masterclass or 4-Part Series	Familiar with basic K8s YAML	Template dry-run demos drive watch time	Lightweight, multi-node, zero cost

4-Week Video Production & Delivery Timeline

EXECUTION WORKFLOW

Phase	Focus Area	Key Actions & Deliverables	Milestone
Week 1	Course Scoping & Lab Build	Create sample raw YAML repo (Deployments, Services, ConfigMaps) and architect transition to a parameter-driven Custom Helm Chart. Set up Kind cluster.	Lab manifests ready
Week 2	Storyboards & Motion Assets	Design animated slides illustrating: Release lifecycle state engine, Go template interpolation pipeline, and OCI registry chart distribution.	Visual deck completed
Week 3	Studio Screencast Capture	Record 1440p 60fps dual-pane terminal screencasts showing side-by-side YAML templates and `helm template --debug` rendered outputs.	A-Roll & B-Roll locked
Week 4	Post-Production & Launch	Cut zoomed inserts for Go template syntax (`{{ .Values }}`), generate 3 high-contrast CTR thumbnails, and push structured Git branches.	Published & Promoted

Recording Rig, Terminal Layout & Retention Engineering

PRODUCTION QUALITY

IDE & Screencast Configuration

- **VS Code Extensions:** Install Helm-Intellisense and YAML extensions for clean autocompletion.
- **Font Hierarchy:** JetBrains Mono at 18pt–20pt with 1.45 line-spacing to guarantee readability on mobile.
- **Dual Terminal Layout:** Left: Template manifest editor; Right: Live watcher with `helm list` and `k9s`.
- **CLI Prompts:** Minimal shell prompt showing active K8s context (e.g., `kind-helm-lab`) and namespace.

YouTube Retention Strategy

- **The Hook (First 45s):** Show a messy folder of 15 hardcoded YAML manifests vs 1 clean command: `helm install my-app ./chart`.
- **Instant Value:** Clarify Helm 3 architecture early (no Tiller security hazard).
- **Live Rollback Drama:** Intentionally release a broken container tag and restore zero-downtime health using `helm rollback` in 10 seconds.
- **Lead Asset:** Provide a downloadable 1-page Helm Go Templating Cheat Sheet in pinned comments.

Pedagogical Core Principle: "Rendered Output Before Cluster Apply"

Never run `helm install` directly after editing a Go template. Always demonstrate `helm template ./chart -f values.yaml` first. Showing the exact interpolated Kubernetes YAML before it hits the API server eliminates the mystery of template engine behavior.

Curriculum Breakdown: Modules 1 to 3

FOUNDATIONS TO CUSTOM CHARTS

Module 1: Why Helm? Architecture, Mental Model & Package Mechanics

Timing: 18 Mins

- **The Problem with Raw K8s Manifests:** Duplicated YAML, copy-paste errors across dev/staging/prod, and lack of release tracking.
- **What is Helm?:** The package manager for Kubernetes (equivalent to apt/brew/npm for cluster resources).
- **Helm 3 Modern Architecture:** Why Helm 2's in-cluster `Tiller` daemon was removed; RBAC security delegation directly through user kubeconfig.
- **Core Terminology:** Chart (package definition), Release (running instance in cluster), Repository (shared catalog of charts).
- **Using Public Registries:** Adding repos (`helm repo add`), updating indices, searching Artifact Hub, and inspecting values via `helm show values` .

Module 2: Managing Release Lifecycles & Atomic Rollbacks

Timing: 20 Mins

- **Installing Charts:** Executing `helm install <release-name> <chart> --namespace <ns> --create-namespace` .
- **Inspecting Active Releases:** Tracking statuses with `helm list` , viewing rendered cluster manifests with `helm get manifest` .
- **Upgrading Workloads:** In-place upgrades using `helm upgrade` and value overriding with `--set` vs `-f custom-values.yaml` .
- **Atomic Safety Flags:** Protecting production using `--atomic` and `--timeout` (automatic rollback if health checks fail).
- **History & Rollback:** Inspecting release revisions with `helm history` and triggering instantaneous rollbacks with `helm rollback <release> <revision>` .
- **Clean Uninstalls:** Removing workloads safely with `helm uninstall` without leaving orphaned resources.

Module 3: Authoring Custom Helm Charts from Scratch

Timing: 25 Mins

- **Scaffolding a Chart:** Dissecting `helm create my-chart` : understanding `Chart.yaml` , `values.yaml` , `templates/` , and `.helmignore` .
- **Stripping Boilerplate:** Cleaning down the generated default files into a minimal, understandable Deployment + Service skeleton.
- **Chart Metadata:** Differentiating `version` (Chart semver packaging) vs `appVersion` (underlying Docker container image tag).
- **Linting & Local Dry-Runs:** Catching YAML indentation syntax errors early using `helm lint` and `helm install --dry-run` .

Creator Visual Cue: The Release State Graph

Show how Helm stores release metadata as Kubernetes Secrets directly inside the target namespace (`sh.helm.release.v1.my-app.v1`). This proves to viewers that Helm doesn't need an external database to track cluster history.

Curriculum Breakdown: Modules 4 & 5

TEMPLATES, SUBCHARTS & OCI

Module 4: Mastering Go Templating, Sprig Functions & Partials

Timing: 25 Mins

- **Built-in Objects:** Accessing `.Values`, `.Chart`, `.Release`, and `.Capabilities` inside manifest templates.
- **Template Directives:** Conditionals (`if / else / end`), iterating lists/maps with `range`, and scoping context with `with`.
- **Sprig Template Functions:** Practical string formatting, quote wrapping with `quote`, default fallbacks with `default`, and base64 encoding for Secrets.
- **Whitespace & Indentation Traps:** The crucial role of whitespace trimming dashes (`{{-` and `-}}`) and the indispensable `nindent` function.
- **Named Templates & Partials:** Defining reusable helper snippets inside `_helpers.tpl` with `{{ define }}` and including them with `include`.

Module 5: Subcharts, Dependencies & OCI Registries (GHCR / Docker Hub)

Timing: 20 Mins

- **Chart Dependencies:** Declaring external dependencies (e.g., PostgreSQL or Redis) inside `Chart.yaml` under `dependencies:`.
- **Dependency Management:** Fetching subcharts with `helm dependency update` and managing local lockfiles (`Chart.lock`).
- **Global Values:** Passing configuration values from parent umbrella charts down to nested subcharts using `global:` keys.
- **Packaging & OCI Registries:** Bundling charts with `helm package`, logging into GHCR/Docker Hub, and pushing charts as OCI artifacts (`helm push my-chart.tgz oci://...`).

YouTube Packaging, Titles, SEO & Promotion Strategy

GROWTH & DISTRIBUTION

Titles, Packaging & SEO

• **Title Option 1 (High CTR):** "Helm Full Course 2026: The Kubernetes Package Manager Masterclass"

• **Title Option 2 (Problem/Solution):** "Stop Writing Raw Kubernetes YAML! Master Helm in 1 Hour"

• **Target Search Queries:** helm tutorial, kubernetes helm crash course, helm templating, helm values yaml, helm rollback.

Thumbnail Strategy & Growth Assets

• **Thumbnail Visual:** Helm steering wheel logo + K8s ship logo + high-contrast text: "K8S TEMPLATES MADE EASY" or "NO MORE COPY-PASTE".

• **Companion Assets:** GitHub repo with `starter-yaml` and `completed-helm-chart` branches linked in description.

• **Shorts/Clips Repurposing:** Cut 3 clips: "Why Helm 3 removed Tiller", "The whitespace nindent trick", "Atomic rollbacks in 30s".

Creator Action Checklist: The Interactive Starter Chart

Provide a GitHub repository with an un-templated microservice deployment. Have students convert hardcoded ports, replica counts, and image tags into dynamic parameters alongside you during the video. This active learning approach produces record-high viewer completion rates.